

Artifact Description

The artifact is an Android inventory management application that I created in CS 360 Mobile Architecture and Programming and it was also developed as the final project for that course. The app was built in Java using Android Studio and allows users to create accounts, log in, and manage inventory items using CRUD which involves create, read, update, and delete functions. You are able to add any item you want on the inventory screen and the app stores data using a SQLite database and includes an alert feature when an item reaches a quantity of zero.

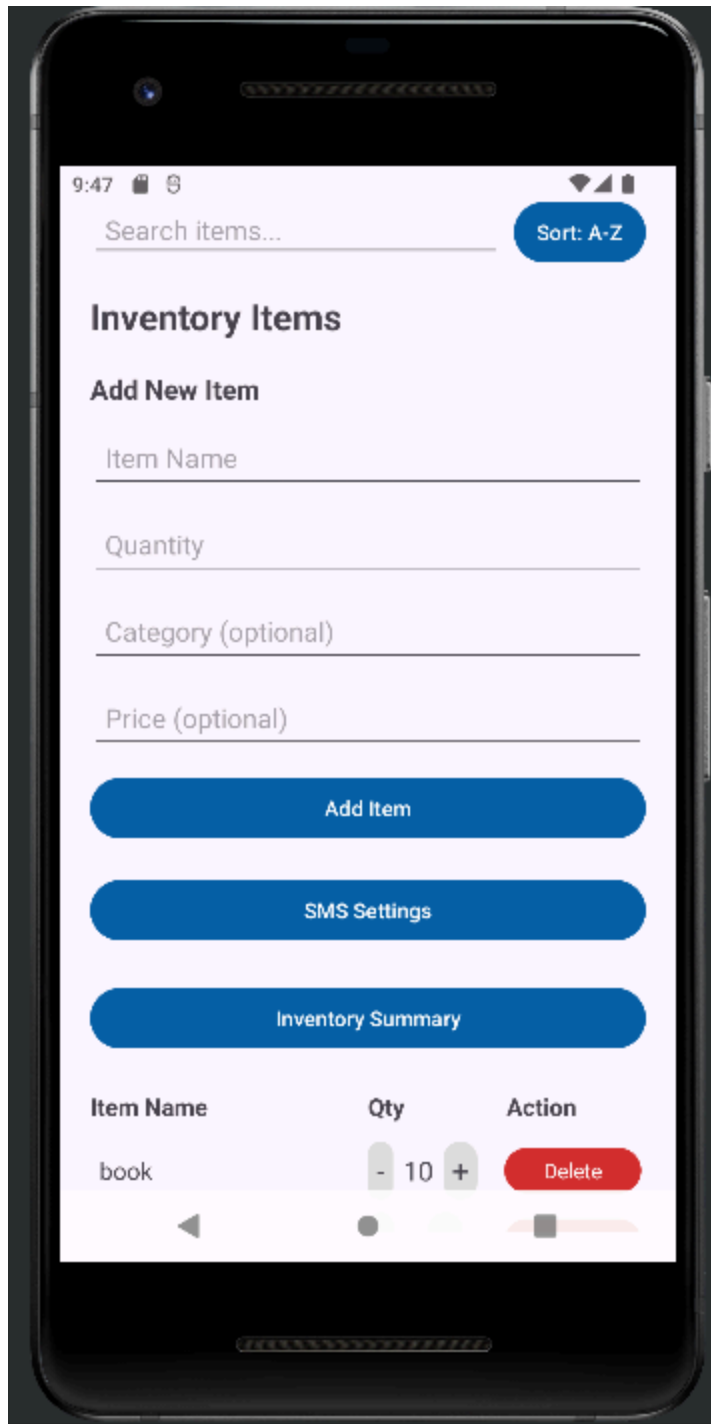
Artifact Selection and Enhancements

I selected this artifact for the databases category because the original version had several significant issues with how data was stored and managed. Passwords were being stored as plain text in the users table, the inventory table only had basic fields with no category, price, or timestamp information, and the database upgrade method was set up in a way that would permanently delete all user data whenever the app's database structure changed. These were real problems that needed to be fixed to bring the database design closer to a professional standard.

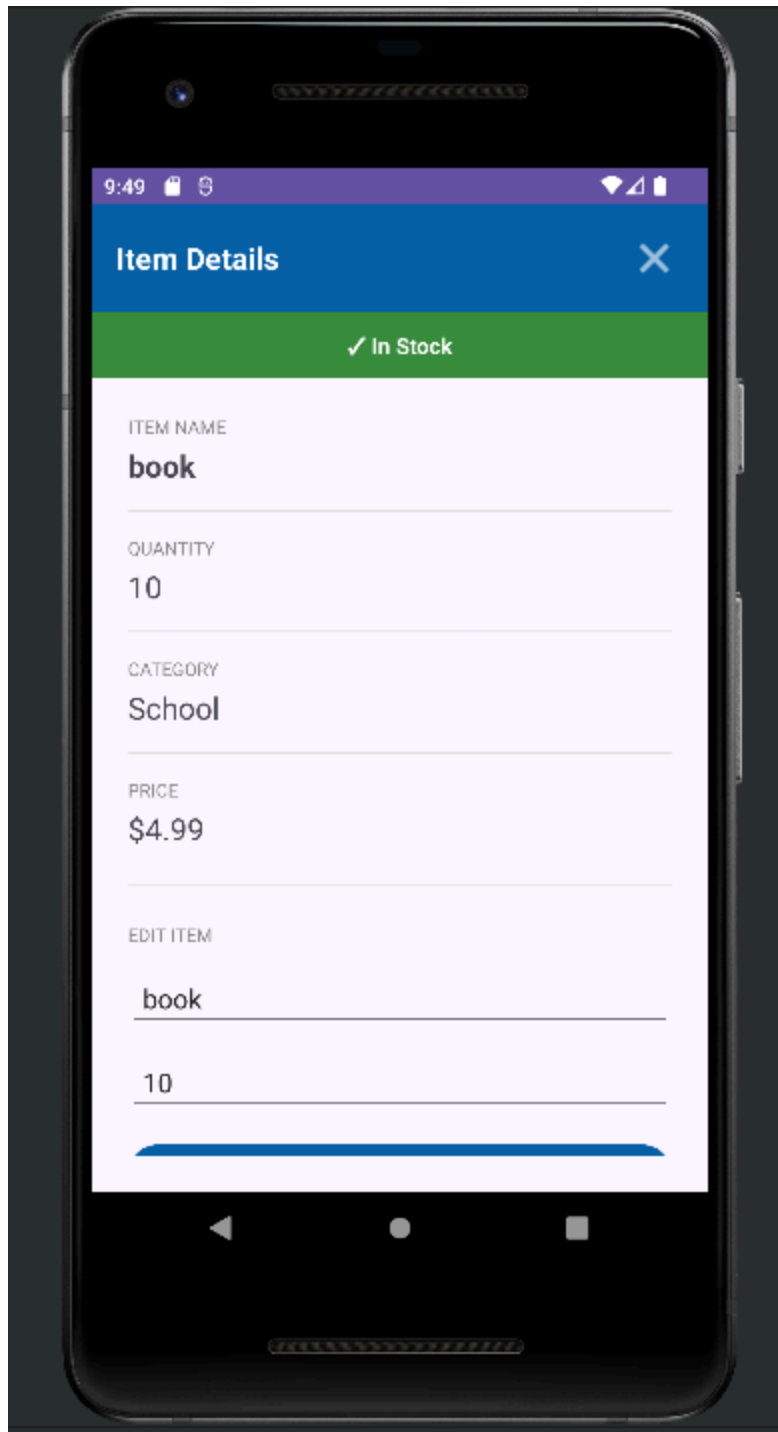
The first improvement was expanding the inventory table schema by adding category, price, and timestamp fields. These additions allow each item to store more detailed information and support features added in Categories One and Two. The second improvement was fixing the database upgrade method. The original version used DROP TABLE, which would permanently delete all user accounts and inventory data whenever the database structure changed. This was replaced with ALTER TABLE so new columns can be added while keeping existing data intact. The third improvement was implementing password hashing so that passwords are never stored as plain text in the database. Instead, the app stores a hashed value and compares hashes during login, which improves security. The fourth improvement was adding an Inventory Summary screen that uses SQL aggregate functions such as COUNT, SUM, and MAX to generate real-time statistics from the database. This screen displays information such as the total number of items, out-of-stock items, low-stock items, total inventory value, the most expensive item, and the most stocked item. Together, these improvements enhanced the database structure, improved security, and demonstrated the ability to use stored data to generate useful information.

Below are screenshots of the enhanced inventory application that demonstrate the improvements made during this milestone.

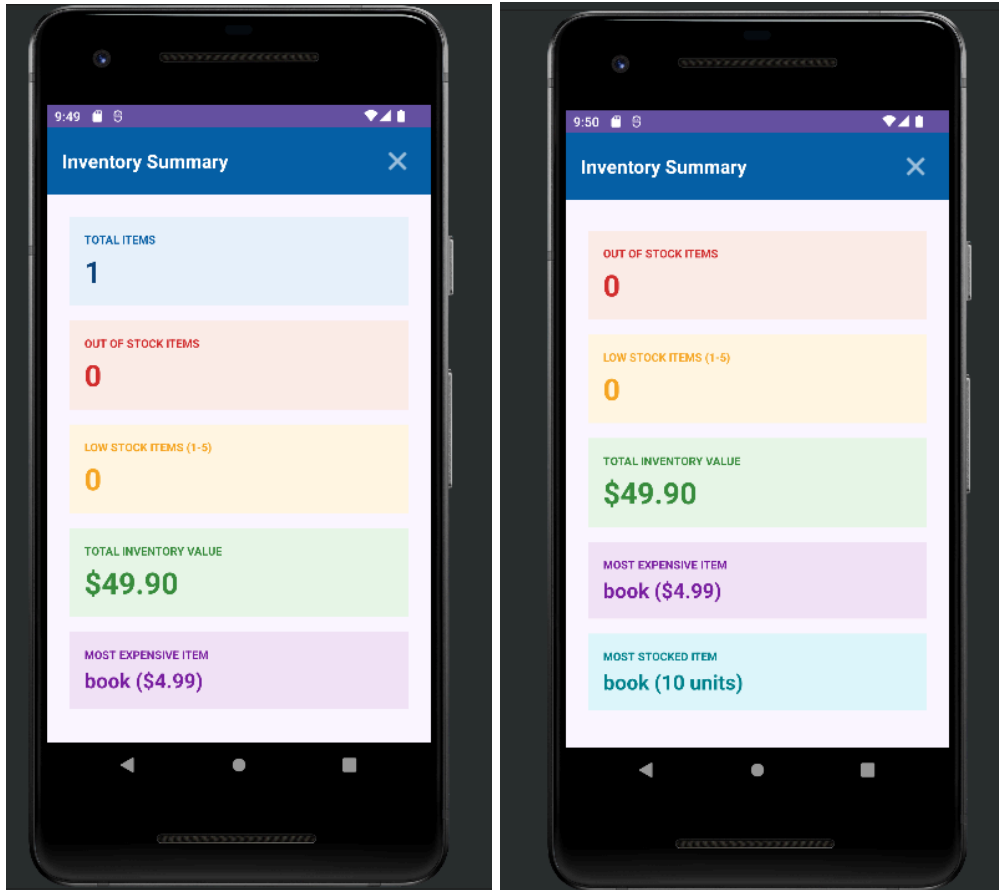
Screenshot 1: Add item form showing the new category and price input fields



Screenshot 2: Item detail screen showing real category and price data



Screenshot 3: Inventory Summary screen showing total items, out of stock count, total value, most expensive item, and most stocked item



Screenshot 4: DB Browser showing the hashed password stored in the users table confirming plain text passwords are no longer saved

Database Structure Browse Data Edit Pragmas Execute SQL			
Table: users			
	id	username	password
	Filter	Filter	Filter
1	1	TestUser377	78ae9e80128180ee8bffa49eb62e19cf4698...

Database Structure Browse Data Edit Pragmas Execute SQL						
Table: inventory Filter in any column						
	id	name	quantity	category	price	created_at
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	book	10	School	4.99	2026-06-06 17:11:04

Course Outcomes

Yes this enhancement met the course outcomes planned in Module One. Course Outcome 3 was met because the database schema design and migration strategy demonstrate the ability to design solutions using proper standards while managing trade offs between simplicity and data safety. Course Outcome 4 was met because the Inventory Summary screen uses advanced SQL aggregate functions to turn inventory data into useful information, which goes beyond basic CRUD and delivers real analytical value. Course Outcome 5 was also met because the password hashing directly addresses the plain text password vulnerability in the original app, and the fixed upgrade method protects against permanent data loss. As of right now there are no updates to my outcome coverage plans. All three course outcomes targeted for this enhancement have been met and the remaining work will be completed in the final ePortfolio submission.

Reflection

One of the most important things I learned from this enhancement was how database schema changes need to be handled carefully in a real application. When I first changed the database structure I had to wipe the app data on the emulator to start fresh because the old schema was still saved on the device. This made me realize why the onUpgrade method is so important and in a real app with real users you cannot just delete everything when the structure changes. Learning how to use ALTER TABLE to add new columns while keeping existing data intact was a practical skill that I did not fully appreciate before working through this enhancement.

Another thing I learned was how SQL aggregate functions like COUNT, SUM, and MAX can be used to extract meaningful information from data that is already being stored. Building the Inventory Summary screen showed me that a well designed database is not just for storing and retrieving individual records and it can also be used to answer bigger questions about the data as a whole.

One challenge I faced was making sure the password hashing worked correctly on both account creation and login. If the hashing process was applied differently in one place than the other the login would always fail. Testing this carefully and verifying the results in DB Browser helped me confirm that the stored hash matched what the login was generating.